

Remote Shutdown System Documentation

Project: Ropo — Remote Shutdown Subsystem

Folder: shutdown/

Last updated: June 2026

1. Folder Purpose

The `shutdown/` folder provides a **safe remote shutdown mechanism** for the Ropo robot. It allows an operator to trigger a full system shutdown of the on-board Raspberry Pi through a mobile-friendly web interface, without requiring physical access to the robot. The architecture separates the control plane (cloud API) from the execution plane (on-device polling client), ensuring that even if the network is intermittent, the shutdown command will eventually be delivered.

2. Architecture Overview

```
Mobile App
  &#9660; POST /shutdown
Railway Cloud (FastAPI)
  &#9660; sets shutdown_requested = true
Raspberry Pi (polls GET /shutdown-status every 15s)
  &#9660; detects shutdown == true
      sudo shutdown -h now
```

The system consists of three tiers:

1. **Cloud API** — a FastAPI application deployed on Railway that exposes a REST endpoint to request a shutdown, protected by a shared bearer token.
2. **Polling Client** — a Python script running on the Raspberry Pi that continuously polls the API every 15 seconds and executes the system shutdown command when instructed.

3. **systemd Service** — ensures the polling client starts at boot and restarts automatically on failure.

Key design decision: The system uses *polling* (client-initiated) rather than push notifications so the Raspberry Pi never needs to expose an open port. This eliminates a major attack surface and simplifies firewall / NAT traversal requirements when the robot operates on customer networks.

3. Files Overview

File	Role	Key Details
<code>main.py</code>	FastAPI backend (165 lines)	REST API, embedded HTML dashboard, token auth, in-memory state
<code>robot_shutdown_client.py</code>	RPi polling daemon (72 lines)	15s poll loop, never-crash design, auto-reset + shutdown
<code>ropo-shutdown.service</code>	systemd unit file	Runs as root, auto-restart, loads <code>/etc/ropo-shutdown.env</code>
<code>requirements.txt</code>	Python dependencies	<code>fastapi</code> , <code>uvicorn</code> , <code>python-dotenv</code>
<code>README.md</code>	Quick-start guide	Setup, environment variables, deployment instructions
<code>__init__.py</code>	Package marker	Empty — enables Python package imports

4. Detailed File Explanations

4.1 — `main.py` — FastAPI Backend

The backend exposes three REST endpoints and serves a self-contained HTML dashboard:

Endpoint	Method	Description
<code>/</code>	GET	Returns the HTML dashboard page (<code>text/html</code>)

Endpoint	Method	Description
<code>/shutdown-status</code>	GET	Returns <code>{"shutdown": true false}</code> — the current state
<code>/shutdown</code>	POST	Accepts <code>{"token": "..."} ; sets <code>shutdown_requested = true</code></code>
<code>/shutdown-reset</code>	POST	Accepts <code>{"token": "..."} ; resets state to <code>false</code></code>

Token validation: The `SHUTDOWN_TOKEN` environment variable is loaded via `python-dotenv` and compared against the `token` field of incoming POST requests. If they do not match, the server returns HTTP 403 — *Unauthorized*. This shared secret prevents unauthorised actors from triggering a shutdown.

In-memory state: A module-level global variable `shutdown_requested` holds the current shutdown flag. It starts as `False`, is set to `True` via `POST /shutdown`, and is reset by the client after it reads the flag. This approach is intentionally simple — no database, no Redis — because the flag is ephemeral and the polling window (15 seconds) is short enough that a single in-memory boolean suffices for a single-robot deployment.

4.2 — HTML Dashboard

The dashboard is embedded directly inside `main.py` as a raw Python string rendered at `GET /`. Key features:

- **Dark theme** — professional, low-glare UI suitable for mobile use in varied lighting.
- **Token input field** — the operator enters the shared secret before issuing commands.
- **Status indicator** — a coloured badge (green/red) showing whether shutdown has been requested.
- **Shutdown / Reset buttons** — send POST requests with the entered token to the respective endpoints.
- **5-second polling** — the dashboard auto-refreshes the status via `fetch('/shutdown-status')` every 5 seconds, providing near-real-time feedback after a client action.

By bundling the front-end inside the backend, the system eliminates the need for a separate web server, CORS configuration, or deployment artefact for the UI — a single `uvicorn` process serves everything.

4.3 — `robot_shutdown_client.py` — Polling Client

This script runs continuously on the Raspberry Pi and implements a **never-crash** polling loop:

```
while True:
    try:
        resp = requests.get(url + "/shutdown-status", timeout=10)
        if resp.json().get("shutdown"):
            requests.post(url + "/shutdown-reset", json={"token": token})
            subprocess.run(["sudo", "shutdown", "-h", "now"])
    except Exception:
        pass                # log and continue
    time.sleep(15)
```

Critical design properties:

- **Graceful error handling** — every network error, timeout, or malformed response is caught by the blanket `except Exception` handler. The client logs the error and retries on the next cycle. It never exits.
- **Reset-before-shutdown** — before issuing the system shutdown command, the client first calls `/shutdown-reset` to clear the flag. This prevents a second (duplicate) poll cycle from attempting another shutdown and ensures the API returns to its idle state for the next command.
- **Configuration** — the API base URL and shutdown token are read from environment variables (`API_URL` , `SHUTDOWN_TOKEN`), matching the systemd `EnvironmentFile` .

4.4 — `ropo-shutdown.service` — systemd Unit

Directive	Value	Purpose
<code>After</code>	<code>network-online.target</code>	Wait for full network connectivity before starting
<code>ExecStart</code>	<code>/usr/bin/python3 /opt/ropo-shutdown/robot_shutdown_client.py</code>	Command to launch the polling client
<code>Restart</code>	<code>always</code>	Automatically restart if the process exits or crashes
<code>RestartSec</code>	<code>5</code>	Wait 5 seconds before restarting

Directive	Value	Purpose
User	root	Required to execute <code>sudo shutdown</code>
EnvironmentFile	/etc/ropo-shutdown.env	Load <code>API_URL</code> and <code>SHUTDOWN_TOKEN</code>

The unit file is designed to be placed at `/etc/systemd/system/ropo-shutdown.service` on the Raspberry Pi. After installation, the service is enabled with `systemctl enable ropo-shutdown.service` so it starts automatically at boot.

5. Code Walkthrough

Screenshot 1: FastAPI endpoints — `request_shutdown` and `reset_shutdown`
Both validate the token, then toggle the global `shutdown_requested` boolean.

Screenshot 2: Embedded HTML dashboard — token form, status badge, Shutdown / Reset buttons
The dark-themed UI with the 5-second polling loop using `setInterval` .

Screenshot 3: Polling client main loop — GET `/shutdown-status`, reset, execute shutdown
The never-crash `while True` pattern with blanket exception handling.

Screenshot 4: *systemd unit file — `ropo-shutdown.service`*

Restart policy, environment file loading, and root user execution.

Screenshot 5: *Railway deployment — environment variables configuration*

Setting `SHUTDOWN_TOKEN` and the start command `uvicorn main:app`.

Screenshot 6: *End-to-end test — mobile app view showing shutdown confirmed*

The dashboard showing "Shutdown Requested" after a successful POST.

6. Sequence Flow

Operator (Mobile App)

1. Opens dashboard URL
2. Enters token
3. Clicks SHUTDOWN

↓ `POST /shutdown {token: "..."}`

FastAPI (Railway)

4. Validates token
5. Sets `shutdown_requested = True`
6. Returns `{"ok": true}`

↓ `200 OK`

Client (RPi – polls every 15 s)

7. `GET /shutdown-status` → `{shutdown: true}`
8. `POST /shutdown-reset` (clear flag)
9. `subprocess.run(["sudo", "shutdown", "-h", "now"])`

↓

Raspberry Pi

10. System halts → power-off

Safety detail: The reset-before-shutdown pattern (step 8) is essential. If the client instead shut down without resetting, the next API poll would see `shutdown: true` again — but the system would already be powering off, making the duplicate harmless. More importantly, resetting *first* means that if the shutdown command itself fails (e.g., insufficient privileges), the system returns to a safe idle state rather than remaining in a perpetual "shutdown requested" condition.

7. Folder Integration

The `shutdown/` folder is a **self-contained, isolated subsystem** within the Ropo project. None of the existing Ropo modules are modified or depend on it. The integration points are purely operational:

- **Deployment:** The `main.py` API is deployed separately on Railway as its own service.
- **Configuration:** The Raspberry Pi reads `/etc/rope-shutdown.env` — a file created during initial setup, not managed by the Ropo build system.
- **Orchestration:** The systemd unit file is installed manually (or via Ansible) on the target Pi. It is independent of the robot's main application lifecycle.

This decoupling means the shutdown system can be developed, tested, and deployed independently. A failure in the shutdown module (e.g., the API is unreachable) has zero impact on the robot's core functionality.

8. Technical Notes

Area	Details
Framework	FastAPI — chosen for async support, automatic OpenAPI docs, and lightweight deployment via Uvicorn. The API runs as a single process with no external database.
Deployment	Railway.app — the service is deployed as a container from a GitHub-connected repository. The start command is <code>uvicorn main:app --host 0.0.0.0 --port 8080</code> . <code>SHUTDOWN_TOKEN</code> is set in Railway's environment variable dashboard.
Polling interval	15 seconds — balances responsiveness with network and CPU overhead. The client uses <code>time.sleep(15)</code> between cycles, so the effective interval is 15 s plus the API round-trip time (typically < 500 ms).
Security	Single shared token transmitted over HTTPS. No TLS termination at the application layer — Railway provides HTTPS at the edge. The token should be a high-entropy string (recommended: 32+ random characters generated via <code>openssl rand -hex 32</code>).
Resilience	The polling client and systemd unit together provide a robust "set and forget" daemon. If the API is temporarily down, the client logs the error and retries. If the client process crashes, systemd restarts it within 5 seconds.

9. Academic Explanation

9.1 — Remote Control Design Pattern

The remote shutdown system follows the **Command** design pattern applied at the system-administration level. A command object (the shutdown request) is created by the operator, serialised as an HTTP POST payload, transmitted over the network, and finally executed by the Raspberry Pi. The state is intentionally ephemeral — once the command has been consumed, the flag is cleared — making this an *execute-once* remote procedure call rather than a persistent state machine.

9.2 — Polling vs. Push

Two fundamental approaches exist for remote device control:

- **Push (WebSocket, MQTT, gRPC stream):** The server initiates communication and sends commands as soon as they are issued. This provides lower latency but requires the device to maintain an open connection and listen on a port or connect to a broker. On a customer's local network, firewalls and NAT may prevent inbound connections.
- **Polling (HTTP Pull):** The device periodically checks a well-known URL for new commands. This adds latency proportional to the polling interval (here, up to 15 seconds) but eliminates the need for inbound ports or persistent connections. It is simpler, more robust in the presence of network interruptions, and easier to debug.

The polling approach was chosen for this system because reliability and security across heterogeneous customer networks were prioritised over sub-second response time. A 15-second shutdown delay is acceptable for a safety-critical remote shutdown.

9.3 — Security via Shared Token

The system employs a **pre-shared key** (PSK) authentication model — the simplest form of symmetric authentication. Both the API server and the polling client know the same secret token. The operator provides it through the web dashboard; the client embeds it in its reset request. Security relies on:

- **Secrecy:** The token is never logged, never sent over unencrypted connections (HTTPS enforced by Railway), and is stored in a restricted `EnvironmentFile` on the Pi.
- **Entropy:** A sufficiently long random token (256 bits) makes brute-force attacks infeasible within the rate-limited context of a single FastAPI instance.
- **No session management:** Because the system has no user accounts or session cookies, the attack surface is limited to token leakage or replay. HTTPS and short polling intervals minimise the window of vulnerability.

9.4 — Safety Considerations

Remote shutdown carries inherent risk: an accidental trigger could halt the robot during operation, potentially causing data loss or unsafe physical states. The system mitigates this through:

- **Token gating:** Only an operator who knows the token can issue a shutdown.
- **Reset-before-shutdown:** The client clears the flag before executing the system halt, ensuring that a single trigger corresponds to a single shutdown action.

- **Polling delay:** The 15-second window gives the operator an opportunity to reset the flag via the dashboard if the shutdown was issued in error, before the client reads it.
- **Degraded behaviour:** If the API is unreachable, the client does nothing — it never assumes a shutdown should occur. The system defaults to *not* shutting down.

10. Conclusion

The Remote Shutdown System is a lightweight, secure, and resilient mechanism for powering off the Ropo robot from a remote location. By combining a cloud-hosted FastAPI endpoint with an on-device polling client managed by systemd, the system achieves a clean separation of concerns: the API handles authorisation and state, while the client handles reliable delivery and execution. The architecture is deliberately simple (a single boolean, a shared token, a 15-second poll loop) because complexity is the enemy of reliability in safety-critical control paths. The result is a production-ready subsystem that can be deployed in minutes and trusted to operate unattended for months at a time.

— End of Document —